

Design & Analysis of Algorithms

Assignment-3

Submitted by

Muhammad Moiz Ansari [23i-0523]

Semester-5

Submitted to

Ms. Noor-ul-Ain



**Department of Computer Science National University of
Computer and Emerging Sciences Islamabad, Pakistan**

September 2025

Contents

Problem 1	4
Decision Criteria	4
Time Complexity.....	4
Example Usage	5
Problem 2.....	6
Pseudocode.....	6
Time Complexity.....	7
Example Usage	8
Problem 3.....	10
Memory-Optimized Design	10
• Compressed Trie	10
• Lazy Failure Link Construction.....	10
• Sparse Node Representation.....	10
• Output Compression	10
Divide and Conquer Construction.....	10
• Divide	10
• Conquer	11
• Search	11
Aho-Corasick Implementation.....	11
• Time Complexity	11
• Example Usage	11
Comparison of Standard & Divided Aho-Corasick	12
• Time Complexity	12
• Space Complexity	13
Pseudocode for Partial Automaton.....	13
Problem 4.....	15
Designing Algorithm	15
• Divide Phase	15
• Conquer Phase	15

• Merge Phase.....	16
• Combine Phase.....	16
Correctness of Merge Step	16
Time Complexity.....	17
Space Complexity.....	17
Comparison with Suffix-array-based Compression	18
Example	18

Problem 1

Decision Criteria

Condition Level	Criteria	Reason / Meaning	Algorithm Chosen
Level 1 - Very Small Pattern or Small Text	$m \leq 5$ OR $n \leq 50$	Short patterns and short texts -> overhead of KMP or hashing not worth it.	Naïve
Level 2 - Medium Pattern ($6 \leq m < 20$)	periodic = true	Repetitive text -> Rabin-Karp produces many collisions	KMP
	$\sigma \leq 6$	Small alphabet → Rabin-Karp collision probability is higher	KMP
	Else ($\sigma > 6$ AND non-periodic)	Randomized text → Rabin-Karp performs fast	Rabin-Karp
Level 3 - Long Pattern ($m \geq 20$)	$\sigma \leq 6$	Small alphabet + long pattern -> KMP performs better	KMP
	$\sigma > 6$ AND periodic = true	Large alphabet + repetitive text -> Rabin-Karp collisions increase	KMP
	$\sigma > 6$ AND periodic = false	Long pattern + diverse alphabet + non-repetitive text -> Rabin-Karp is best	Rabin-Karp
Default Case	No rule matched	A fallback to all above cases (just for safety)	Rabin-Karp

Where, sigma denotes the number of unique characters used in text and pattern.

Time Complexity

n = Length of text string

m = Length of pattern string

Function	Best case	Average Case	Worst Case
Calculating sigma	$O(n)$	$O(n)$	$O(n)$
isPeriodic (const string& T)	$O(m)$	$O(m)$	$O(m)$
matcherSelector (const string& T, const string& P, int sigma)	$O(1)$	$O(1)$	$O(1)$
naiveStringMatcher (const string& T, const string& P)	$O(n)$	$O(m(n-m+1))$	$O(m(n-m+1))$
rabinKarp (const string& T, const string& P, int d, int q)	$O(n+m)$	$O(n+m)$	$O(m(n-m+1))$
computePrefixFunction (const string& P)	$O(m)$	$O(m)$	$O(m)$
kmpMatcher (const string& T, const string& P)	$O(n)$	$O(n)$	$O(n)$

Example Usage

- Example 1 (Small pattern)

Text: hellotheregeneral

Pattern: he

> Naive String Matcher <

Pattern occurs with shift 0

Pattern occurs with shift 6

- Example 2 (medium pattern + periodic text or small alphabet)

Text: ababababababababababababcdxyz

Pattern: abababab

> KMP String Matcher <

Pattern occurs with shift 0

Pattern occurs with shift 2

Pattern occurs with shift 4

Pattern occurs with shift 6

Pattern occurs with shift 8

Pattern occurs with shift 10

Pattern occurs with shift 12

Pattern occurs with shift 14

Pattern occurs with shift 16

- Example 3 (medium pattern + large alphabet + non-periodic)

Text: QWERTYUIOPLKJHGFDSA ZXC VBNM the quick brown fox jumps over the lazy dog

Pattern: quickbrown

> Rabin Karp String Matcher <

Pattern occurs with shift 29

Problem 2

Pseudocode

```
// Phase 1: Divide
```

```
Input: Document A, Document B
```

```
K = Chunk size
```

```
n = Length of A
```

```
m = Length of B
```

```
// Calculate number of chunks
```

```
numChunksA =  $\lceil n/k \rceil$ 
```

```
numChunksB =  $\lceil m/k \rceil$ 
```

```
// Create chunk boundaries
```

```
For i = 0 to numChunksA-1:
```

```
    chunkA[i] = A[i*k : min((i+1)*k, n)]
```

```
For j = 0 to numChunksB-1:
```

```
    chunkB[j] = B[j*k : min((j+1)*k, m)]
```

```
// Phase 2: Conquer
```

```
For each chunk pair (chunkA[i], chunkB[j]):
```

```
    // Create DP table
```

```
        Let m = length(chunkA[i])
```

```
        Let n = length(chunkB[j])
```

```
        Create table dp[m+1][n+1]
```

```
// Fill DP table
```

```
    For row = 1 to m:
```

```
        For col = 1 to n:
```

```
            if chunkA[i][row-1] == chunkB[j][col-1]:
```

```

        dp[row][col] = dp[row-1][col-1] + 1
        if dp[row][col] > maxLength:
            maxLength = dp[row][col]
            record position
    else:
        dp[row][col] = 0

// Update global maximum
    if found length > globalMax:
        globalMax = new match

// Phase 3: Merge (Handle boundaries as well)
For each boundary point between chunks:
    // Extract overlapping windows
        windowA = A[boundary - windowSize : boundary + windowSize]
        windowB = B[boundary - windowSize : boundary + windowSize]

    // Run LCS on these windows
        result = findLCS(windowA, windowB)

    // Update global max if needed
        if result.length > globalMax:
            globalMax = result

```

Time Complexity

Phase	Operation	Iterations/Calls	Cost per Operation	Time	Space
Phase 1: Divide	Calculate chunks	$O(1)$	$O(1)$	$O(1)$	$O(1)$
Phase 1: Divide	Create chunkA	$[n/k]$	$O(k)$	$O(n)$	$O(n)$

Phase 1: Divide	Create chunkB	$[m/k]$	$O(k)$	$O(m)$	$O(m)$
Phase 1 Total				$O(n + m)$	$O(n + m)$
Phase 2: Conquer	Process chunk pairs	nm/k^2	$O(k^2)$	$O(nm)$	$O(k^2)$
Phase 2: Conquer	DP table creation	per pair	$O(k^2)$	-	$O(k^2)$
Phase 2: Conquer	DP table filling	per pair: $k*k$	$O(1)$ per cell	-	-
Phase 2: Conquer	Update global max	per pair	$O(1)$	$O(nm/k^2)$	$O(1)$
Phase 2 Total				$O(nm)$	$O(k^2)$
Phase 3: Merge	Boundary checks	$(n/k) * (m/k) \approx nm/k^2$	$O(k^2)$	$O(nm)$	$O(k^2)$
Phase 3: Merge	Window extraction	per boundary	$O(k)$	-	$O(k)$
Phase 3: Merge	LCS on windows	per boundary	$O(k^2)$	-	$O(k^2)$
Phase 3: Merge	Update global max	per boundary	$O(1)$	$O(nm/k^2)$	$O(1)$
Phase 3 Total				$O(nm)$	$O(k^2)$
Overall				$O(nm)$	$O(n+m+k^2)$

Example Usage

Input:

Document A: "the quick brown fox jumps over" (31 chars)

Document B: "a quick brown fox is here" (25 chars)

Chunk Size: 10

Execution:

Phase 1: Divide

Document A chunks:

[0]: "the quick " [0:10]

[1]: "brown fox " [10:20]

[2]: "jumps over" [20:31]

Document B chunks:

[0]: "a quick br" [0:10]

[1]: "own fox is" [10:20]

[2]: " here" [20:25]

Chunk pairs to process: $3 \times 3 = 9$ pairs

Phase 2: Conquer

Pair: A[0] vs B[0] -> "the quick " vs "a quick br"

Found: " quick " (length 7) at position (3, 1)

Pair: A[1] vs B[1] -> "brown fox " vs "own fox is"

Found: "own fox " (length 8) at position (2, 0)

Current Best: "own fox " (length 8)

Phase 3: Combine

Critical Boundary: Between A[0]-A[1] and B[0]-B[1]

Boundary position in A: 10

Boundary position in B: 10

Window A: [10-5 : 10+5] = [5:15]

Content: "uick brown"

Window B: [10-5 : 10+5] = [5:15]

Content: "ick brown "

Run LCS on these windows:

Found: "ick brown" (length 9) – Current best

Expanding around this match:

Document A: "the quick brown fox..."

Document B: "a quick brown fox is..."

Actual match: "quick brown fox" (length 15)

Explanation:

Chunk A[0] had "quick " (partial)

Chunk A[1] had "brown fox" (partial)
Only boundary check found the full substring.

Final Result:

Maximum Common Substring Length: 15

Substring: "quick brown fox"

Position in A: 4

Position in B: 2

Context in A: "the quick brown fox jumps"

Context in B: "a quick brown fox is here"

Problem 3

Memory-Optimized Design

- **Compressed Trie**

- Instead of storing one character per node, store whole substring in one node.
- This will reduce the number of nodes, when patterns share long common strings.
- Space complexity: $O(k)$ instead of $O(m)$, where k = Number of patterns.

- **Lazy Failure Link Construction**

- Instead of building all possible links, compute failure links on demand.
- This saves space by computing only the failure links needed according to the given text and patterns.
- Use cache computed links for reuse.
- Space complexity: $O(\text{actually used links})$ instead of $O(\text{all possible links})$.

- **Sparse Node Representation**

- Use arrays only for larger number of unique characters.
- Use hash maps for small number of unique characters.
- Space complexity: $O(\sigma)$, where σ = number of unique characters.

- **Output Compression**

- Store only the unique pattern IDs in outputs.

Divide and Conquer Construction

- **Divide**

Divide the pattern into k groups on the basis on any of the following:

- By first character: E.g. If $k=2$, then Group 1 $\rightarrow$ A-M, Group 2 $\rightarrow$ N-Z
- By pattern length: E.g. Short (<10), Medium (10-20), Long (>20)
- By hash value: $\text{hash}(\text{pattern}) \% k$

- **Conquer**

- Build k separate Aho-Corasick automata
- Each automaton handles $\sim(m/k)$ total pattern length.
- Can be built in parallel using threads or on GPU.

- **Search**

- Each automaton reports its matches independently
- Can be parallelized

Aho-Corasick Implementation

- **Time Complexity**

n = Length of text string

k = Number of patterns

m = Sum of lengths of all pattern strings

Σ = Number of unique characters used in text and pattern.

Z = Number of matches

Function	Best case	Average Case	Worst Case
<code>toLowerCase(string& str)</code> - Preprocessing	$O(n + m)$	$O(n + m)$	$O(n + m)$
<code>buildTrie()</code> - Preprocessing	$O(m)$	$O(m)$	$O(m\Sigma)$
<code>buildFailureLinks(TrieNode* root)</code> - Preprocessing	$O(m)$	$O(m)$	$O(m\Sigma)$
<code>search(const string& text)</code>	$O(n)$	$O(n + Z)$	$O(n\Sigma)$
<code>deleteTrie(TrieNode* node)</code>	$O(m)$	$O(m)$	$O(m)$

- **Example Usage**

Text: She sells SEASHELLS by the seashore. HIS sister says SHE has Hershey's

Patterns:

1. she
2. sea
3. shells
4. HIS
5. hers

6. he
7. sh?
8. h?s
9. s?a

=== String Finding Results ===

Pattern "she" found at shift 0

Pattern "he" found at shift 1

Pattern "sea" found at shift 10

Pattern "she" found at shift 13

Pattern "he" found at shift 14

Pattern "shells" found at shift 13

Pattern "he" found at shift 24

Pattern "sea" found at shift 27

Pattern "sh?" found at shift 30

Pattern "his" found at shift 37

Pattern "she" found at shift 53

Pattern "he" found at shift 54

Pattern "h?s" found at shift 57

Pattern "he" found at shift 61

Pattern "hers" found at shift 61

Pattern "she" found at shift 64

Pattern "he" found at shift 65

Comparison of Standard & Divided Aho-Corasick

- **Time Complexity**

Function	Standard AC	Divided AC (Sequential)	Divided AC (Parallel)
toLowerCase(string& str) - Preprocessing	$O(n + m)$	$O(n + m)$	$O(n + m)$

buildTrie() - Preprocessing	$O(m)$	$O(m)$	$O(m/k)$
buildFailureLinks(TrieNode* root) - Preprocessing	$O(m)$	$O(m)$	$O(m/k)$
search(const string& text)	$O(n + Z)$	$O(kn + Z)$	$O(n + Z)$
deleteTrie(TrieNode* node)	$O(m)$	$O(m)$	$O(m/k)$

- **Space Complexity**

Function	Standard AC	Divided AC (k groups)
Space per automaton	$O(m \Sigma)$	$O((m/k) \Sigma)$
Total space	$O(m \Sigma)$	$O(m \Sigma)$
Nodes in trie	$O(m)$	$O(m)$ total, $O(m/k)$ per automaton
Failure links	$O(m)$	$O(m)$
Cache efficiency	Low (if m is large)	High (Smaller per automaton)

Pseudocode for Partial Automaton

// BUILDING PARTIAL AUTOMATON

BuildDividedAutomaton(patterns[], k):

 // Making k groups

 groups[] = new Array[k]

 for each pattern in patterns:

 groupIndex = hash(pattern[0]) mod k

 groups[groupIndex].add(pattern)

 // Partial automaton per group

 automata[] = new Array[k]

 for i = 0 to k-1:

 automata[i] = new AhoCorasick(groups[i])

 automata[i].buildTrie()

 automata[i].buildFailureLinks()

 return automata

// SEARCHING PARTIAL AUTOMATON

SearchWithDividedAutomata(text, automata[], k):

 results[] = empty list

```

// Search with each automaton
for i = 0 to k-1:
    matches[] = automata[i].search(text)
    results.append(matches)
// Merge results (sort by position)
return sortByPosition(results)

```

// MERGING PARTIAL AUTOMATA

```

MergeAutomata(automaton1, automaton2):
    merged = new TrieNode()
    // Merging tries(trees)
    for each child in automaton1.root.children:
        merged.children.add(child)
    for each child in automaton2.root.children:
        if merged.children.contains(child.character):
            MergeTries(merged.children[child.character], child)
        else:
            merged.children.add(child)
    // Rebuild failure links for merged trie
    merged.buildFailureLinks()
    return merged

```

```

MergeTries(node1, node2):
    // Combine outputs
    node1.output.append(node2.output)
    // Recursively merge children
    for each child in node2.children:
        if node1.children.contains(child.character):
            MergeTries(node1.children[child.character], child)

```

```
else:  
    node1.children.add(child)
```

Problem 4

Designing Algorithm

- **Divide Phase**

- Divide the input string into k sub blocks of equal sizes (can differ very slightly if not equally divisible)
- For optimal performance, block size of $\sqrt{n}$ is recommended.

Example:

String: "abcbcabcbxyzxyz" (length 15)

Divide into 3 blocks:

- Block 1: "abcbab" (length 5)
- Block 2: "cbcbx" (length 5)
- Block 3: "xyzxyz" (length 5)

- **Conquer Phase**

Find repetitions in each block.

For each block independently:

- Build a suffix array or hash table for that block
- Find the longest repeated substring within that block
- Store:
 - The repeated substring
 - Its positions
 - Compression gain (original length - compressed representation)

Example for Block 1 ("abcbab"):

- Repeated substring: "ab"
- Positions: [0, 3]
- "ab" + pointer(back 3, length 2)

- **Merge Phase**

Handle cross-boundary repetitions

A repeated pattern can span multiple blocks. So, to solve that:

- Check boundary conditions (Check last k characters of block I with first k characters of block i+1).
- Use rolling hash or suffix trees to find cross-boundary matches.
- Compare compression gain:
Local repetition in blocks vs Global repetition across blocks.
- Select the best compression strategy.

Merge Algorithm:

For each pair of adjacent blocks:

```
regionOverlap = last_chars(block[i]) +  
first_chars(block[i+1])
```

```
Find longest repeated substring in regionOverlap
```

```
If cross-boundary pattern gives better compression:
```

```
Update compression dictionary
```

- **Combine Phase**

Combining results to produce final compressed representation.

Combine Algorithm:

```
compressed_output = []
```

For each position in original string:

```
If current substring matches a pattern in dictionary:
```

```
compressed_output.add((pointer, length)) // LZ77 style
```

```
Else:
```

```
compressed_output.add(original_character) // Literal  
style
```

Correctness of Merge Step

Without merge:

Original: "abcbabcabc"

Block 1: "abcab" -> finds "ab" repeated

Block 2: "cabc" -> none repeating

Result: Misses the global "abc" pattern

With merge:

Check boundary: "cab|cabc"

Finds "abc" spans both blocks

Achieves better compression: "abc" × 3 instead of "ab" × 2

Correctness Guarantee:

The algorithm is correct because:

1. Local patterns are found within each block (completeness within blocks).
2. Cross-boundary patterns are found in merge phase (completeness across blocks).
3. We choose the best compression.

Time Complexity

n = Length of string

k = Number of blocks

Function	Complexity
DIVIDE	
Dividing blocks	$O(1)$
CONQUER (Per block)	
Building suffix array	$O((n/k)^2 \log(n/k))$
Finding repetitions	$O((n/k)^2)$
Total for all k blocks	$O(n^2/k)$
MERGE (Checking k-1 boundaries)	
For each boundary	$O((n/k)^2 \log(n/k))$
Total	$O(n^2/k)$
COMBINE	
Combine	$O(n)$
Overall	$O(n^2/k)$, where $k = \sqrt{n}$ gives us $O(n^{1.5})$

Space Complexity

Function	Complexity
Suffix arrays for each block	$O(n/k)$ per block × k blocks = $O(n)$
Hash tables	$O(n)$

Output	$O(n)$
Total	$O(n)$

Comparison with Suffix-array-based Compression

Aspect	Divide & Conquer	Suffix Array
Time	$O(n^{1.5})$	$O(n^2 \log n)$
Space	$O(n)$	$O(n)$
Parallelization	Easy (Independent blocks)	Hard (Global structure)
Incremental Updates	Easy (update one block)	Hard (rebuild entire array)
Best for	Large texts, streaming data	Small to medium texts

Example

Input String: "TOBEORNOTTOBEORTOBEORN" (24 characters)

Divide (3 blocks):

- Block 1: "TOBEORNO"
- Block 2: "TTOBEOR"
- Block 3: "TOBEORNO"

Conquer (local patterns):

- Block 1: "OR" appears twice
- Block 2: "T" appears three times, "TO" twice
- Block 3: "OR" appears twice

Merge (find cross-boundary):

- Boundary 1-2: "NOTTTO" → finds "T" pattern
- Boundary 2-3: "ORTOBEOR" → finds "TOBEOR" pattern!

Result:

Global pattern found: **"TOBEOR"** repeats 3 times!

Compressed output:

"TOBEORN" + <back 9, len 6> + <back 15, len 9>

Original: 24 bytes

Compressed: ~15 bytes (37% compression)